

Step 4 — Forgetting, and a Privacy-Respecting Delete — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 3's `state.py`, `store.py`, `memory_writer.py`, `graph.py`, and `main.py` already exist and work.

Basic Version

Approach 1 — the direct way, one key at a time, by hand

```
# store.py (Basic -- deletes one named key, not everything)
from store import get_store, namespace_for

def forget_one_fact(user_id: str, key: str) -> None:
    store = get_store()
    store.delete(namespace_for(user_id), key)
```

Expected output: calling `forget_one_fact("sam", "preferred_name")` removes that one key. But if `"stated_preference"` was also saved, it's still sitting there afterward — this version only forgets what you remembered to name, which isn't really forgetting the user, just one fact about them. This is exactly the gap Intermediate closes.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real `forget_user`, deleting everything found in the namespace

```
# store.py (adds to Step 2/3's get_store / namespace_for)
from langgraph.store.memory import InMemoryStore
```

```
_store = InMemoryStore()
```

```
def get_store():
    return _store
```

```
def namespace_for(user_id: str) -> tuple:
    return (user_id, "memories")
```

```
def forget_user(user_id: str) -> int:
    """Delete every saved fact for this user.
```

A real delete, not a soft "is_forgotten" flag -- once this runs, the facts are gone from the Store, not just hidden from future reads. Returns how many items were deleted, so callers can confirm something actually happened instead of assuming success.

```
"""
```

```
    store = get_store()
    namespace = namespace_for(user_id)
    items = store.search(namespace, query="all facts")
    for item in items:
        store.delete(namespace, item.key)
    return len(items)
```

Expected output:

```
>>> store.put(namespace_for("sam"), "preferred_name", {"text": "Sam"})
>>> store.put(namespace_for("sam"), "stated_preference", {"text": "Likes short an
>>> forget_user("sam")
2
```

```
>>> store.search(namespace_for("sam"), query="anything")  
[]
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Approach 2 — wiring `forget_me` into `main.py`'s loop

```
# main.py (adds the forget_me command to Step 3's loop)
import sqlite3
import uuid
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph
from store import get_store, namespace_for, forget_user

USER_ID = "sam"

def main():
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    store = get_store()
    graph = build_graph(checkpointer, store)
    thread_id = str(uuid.uuid4())

    print(f"MemoryKeeper -- user_id={USER_ID}, thread_id={thread_id}. Type 'exit'")
    while True:
        text = input("you: ")
        if text.strip().lower() == "exit":
            break
        if text.strip().lower() == "forget_me":
            deleted_count = forget_user(USER_ID)
            print(f"bot: All {deleted_count} saved memories for {USER_ID} have been deleted")
            continue # never reaches the graph -- this is a direct, deterministic action

        config = {"configurable": {"thread_id": thread_id}}
        result = graph.invoke(
            {"messages": [{"role": "user", "content": text}], "user_id": USER_ID},
            config=config,
        )
        print("bot:", result["messages"][-1].content)

if __name__ == "__main__":
    main()
```

Expected output:

you: I'm Sam.

bot: Got it, Sam!

you: forget_me

bot: All 1 saved memories for sam have been deleted.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the full honest test: seed, recall, forget, recall again

```
# test_memory_keeper.py (final additions, on top of Steps 1-3's tests)
import sqlite3
import uuid
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph
from store import get_store, namespace_for, forget_user

def ask_as_user(user_id: str, text: str, store) -> str:
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    graph = build_graph(checkpointer, store)
    thread_id = str(uuid.uuid4()) # a brand-new thread, every single call
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke(
        {"messages": [{"role": "user", "content": text}], "user_id": user_id},
        config=config,
    )
    return result["messages"][-1].content

def test_forget_user_actually_emptyies_the_namespace():
    store = get_store()
    namespace = namespace_for("alex")
    store.put(namespace, "preferred_name", {"text": "Alex"})
    store.put(namespace, "stated_preference", {"text": "Likes short answers"})

    deleted_count = forget_user("alex")
    assert deleted_count == 2

    remaining = store.search(namespace, query="anything")
    assert remaining == []

def test_fresh_thread_after_forgetting_has_no_recall():
    store = get_store()
    namespace = namespace_for("jordan")
```

```

store.put(namespace, "preferred_name", {"text": "Jordan"})

# confirm recall works BEFORE forgetting -- otherwise a "no memory" result
# afterward could just mean recall was broken the whole time
before_answer = ask_as_user("jordan", "What's my name?", store)
assert "jordan" in before_answer.lower()

forget_user("jordan")

after_answer = ask_as_user("jordan", "What's my name?", store)
assert "jordan" not in after_answer.lower()

if __name__ == "__main__":
    test_forget_user_actually_emptyies_the_namespace()
    print("PASS: forget_user removes every key in the user's namespace")
    test_fresh_thread_after_forgetting_has_no_recall()
    print("PASS: a brand-new thread, after forgetting, genuinely knows nothing")

```

Expected output:

```

PASS: forget_user removes every key in the user's namespace
PASS: a brand-new thread, after forgetting, genuinely knows nothing

```

The second test is written in three parts on purpose: prove recall works first, forget, then prove recall stopped working. Skipping the first part would leave a real gap in the proof -- a broken `recall_node` and a working `forget_user` would look identical from the outside if you only ever checked the "after" state.

Difference from Intermediate, and what this approach adds: Intermediate proves `forget_user` empties the Store when you call it directly and check the Store yourself, in the same process. Advanced proves the thing a real user actually cares about: ask the assistant something before forgetting, confirm it knew the answer, forget, ask again through the exact same graph-invoke path a real user would use, and confirm the knowledge is genuinely gone -- not inferred from internals, observed the same way the user would observe it.

Which one should you actually write? Write `forget_user` exactly as shown in Intermediate -- search everything in the namespace, delete each item found, return a count so the caller (and your tests) can confirm something real happened. Keep `forget_me` as a direct, non-LLM command in `main.py` -- a delete this consequential should never depend on a model correctly recognizing intent; matching a plain, exact string is the right amount of "smart" for this command. Keep both Advanced tests: the direct Store check is fast and catches a broken `forget_user` immediately, and

the full ask-forget-ask-again test is the one that actually matches the promise this feature makes to a real user -- "ask me to forget you, and I genuinely will."